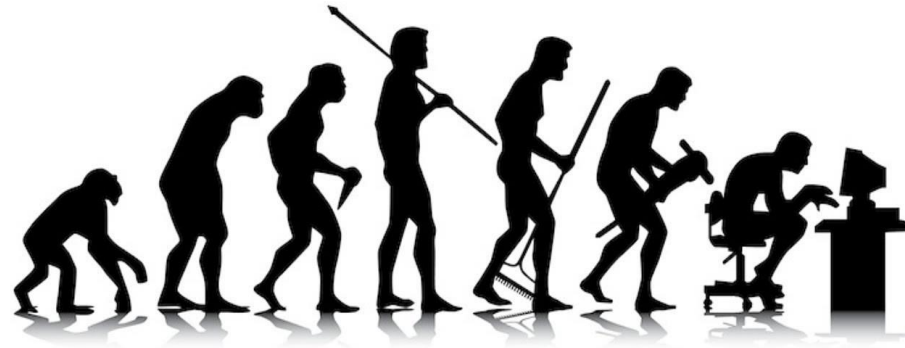


Übungen Streams



Setup: Student

Erstelle eine Klasse oder ein Record **Student**, das du für **alle** nachfolgenden Aufgaben verwendest:

```
public record Student(String name, int age, double avgGrade)
```

Hinweis:

Jedes Student-Objekt hat die Attribute name (String), age (int) und avgGrade (double). Das Attribut avgGrade repräsentiert den Durchschnittsnotenwert von 1 bis 6, wobei 1 die beste und 6 die schlechteste Durchschnittsnote ist.

Setup: StudentManager

Erstelle eine Klasse **StudentManager**, die folgendes Interface **StudentProcessor** implementiert. Lasse jede Methode zunächst Standardwerte zurückgeben (return null oder 0).

```
public interface StudentProcessor {  
  
    Collection<String> getDetails(Collection<Student> students);  
  
    List<String> findNames(Collection<Student> students, double minGrade);  
  
    List<String> findNames(Collection<Student> students , double minGrade, int minAge);  
  
    double averageGrade(Collection<Student> students);  
  
    Optional<Student> findTopStudent(Collection<Student> students);  
  
    boolean exists(Collection<Student> students, String name);  
  
    List<Student> adjustedGrades(Collection<Student> students, double minGrade, int minAge, double adjustment);  
}
```

Student Details

Gegeben ist eine beliebige Collection von Student-Objekten (Parameter).

Implementiere folgende Methode **mittels Stream** in **StudentManager**:

Collection<String> `getDetails`(Collection<Student> students)

Es soll eine Collection von Strings zurückgegeben werden. Jeder String enthält Informationen zum jeweiligen Studenten und ist wie folgt aufgebaut:

Format: name() | age() Jahre | Durchschnittsnote: avgGrade()

Konkret: Hannah | 23 Jahre | Durchschnittsnote: 1.3

Beispiel `System.out.println(studentManager.getDetails(students))`:

[Hannah | 23 Jahre | Durchschnittsnote: 1.3, Heinz | 22 Jahre | Durchschnittsnote: 4.1]

Student Details: Lösung

```
public Collection<String> getDetails(Collection<Student> students) {  
    return students.stream()  
        .map(student -> String.format(  
            Locale.ENGLISH, // Optionaler Parameter, lässt man ihn weg, wird die Systemsprache zur Umwandlung verwendet.  
            "%s | %d Jahre | Durchschnittsnote: %.1f", // Das Zeichen für die Nachkommastelle hängt per default von Systemsprache des Betriebssystems ab.  
            student.name(), student.age(), student.avgGrade()))  
        .toList();  
}
```

```
public Collection<String> getDetails(Collection<Student> students) {  
    return students.stream().map(student ->  
        student.name() + " | " + student.age() + " Jahre | Durchschnittsnote: " + student.avgGrade())  
        .toList();  
}
```

Filtern nach Durchschnittsnote

Gegeben ist eine beliebige Collection von Student-Objekten. Implementiere folgende Methode **mittels Stream** in die Klasse **StudentManager**:

- Filtere alle Studenten heraus, die ein avgGrade größer gleich minGrade haben (z. B. 4,3).
- Gebe nur die Namen als List zurück.
- Die Namen sind alphabetisch aufsteigend sortiert (von A nach Z).

```
public static List<String> findNames(Collection<Student> students, double minGrade)
```

Filtern nach Durchschnittsnote: Lösung

```
public List<String> findNames(Collection<Student> students, double minGrade) {  
    return students.stream()  
        .filter(student -> student.avgGrade() >= minGrade)  
        .map(Student::name)  
        .sorted()  
        .toList();  
}
```

Filtern nach Durchschnittsnote und Alter

Ergänze **StudentManager** um folgende Methode, die **ein Stream** verwendet:

- Filtere alle Studenten heraus, die ein avgGrade größer gleich minGrade haben und mindestens minAge Jahre (inklusive) alt sind.
- Sortiere die Studenten nach Alter aufsteigend.
- Gebe nur die Namen der Studenten als List zurück.

```
public List<String> findNames(Collection<Student> students, double minGrade, int minAge)
```


Filtern nach Durchschnittsnote und Alter: Lösung

```
public List<String> findNames(Collection<Student> students, double minGrade, int minAge) {  
    return students.stream()  
        .filter(student -> student.age() >= minAge && student.avgGrade() >= minGrade)  
        .sorted((student1, student2) -> Integer.compare(student1.age(), student2.age()))  
        // .sorted((student1, student2) -> student1.age() - student2.age())  
        // .sorted(Comparator.comparingInt(Student::age))  
        .map(Student::name)  
        .toList();  
}
```

Gesamtdurchschnitt

Erweitere **StudentManager** um eine Methode, die den Gesamtdurchschnitt der Durchschnittsnoten (avgGrade) aller Studenten **mittels Stream** berechnet.

```
public double averageGrade(Collection<Student> students)
```

Wenn kein Average ermittelt werden kann, soll 0.0 zurückgegeben werden.

Gesamtdurchschnitt: Lösung

```
public double averageGrade(Collection<Student> students) {  
    return students.stream()  
        .mapToDouble(Student::avgGrade)  
        .average()  
        .orElse(0.0);  
}
```

Top-Student

Erweitere **StudentManager** um eine Methode, die den Studenten mit der besten Durchschnittsnote (eine 1 wäre Beste) aus der Collection **mittels Stream** zurückgibt.

```
public Optional<Student> findTopStudent(Collection<Student> students)
```

Wenn kein top Student gefunden werden kann, soll ein leere Optional zurückgegeben werden.

Top-Student: Lösung

```
public Optional<Student> findTopStudent(Collection<Student> students) {  
    return students.stream()  
        .min((student1, student2) -> Double.compare(student1.avgGrade(), student2.avgGrade()));  
  
    // return students.stream()  
    //     .max((student1, student2) -> Double.compare(student2.avgGrade(), student1.avgGrade()));  
  
    // return students.stream()  
    //     .max(Comparator.comparingDouble(Student::avgGrade));  
  
    // return students.stream()  
    //     .sorted(Comparator.comparing(student -> student.avgGrade())).findFirst();  
}
```

Student existiert

Erweitere **StudentManager** um eine Methode, die **mittels Stream** prüft, ob ein Student mit einem bestimmten Namen existiert (ignoriere Groß- und Kleinschreibung).

```
public boolean exists(Collection<Student> students, String name)
```

Student existiert: Lösung

```
public boolean exists(Collection<Student> students, String name) {  
    return students.stream()  
        .anyMatch(student -> student.name().equalsIgnoreCase(name));  
}
```

Notenanpassung

Gegeben ist folgende Methode. Schreibe sie so um, dass nur **ein Stream** verwendet wird. Schreibe deine Lösung in **StudentManager**.

```
public List<Student> adjustedGrades(Collection<Student> students, double minGrade, int minAge, double adjustment) {  
    var result = new ArrayList<Student>();  
    for (var student : students) {  
        if (student.age() >= minAge) {  
            var newStudent = new Student(student.name(), student.age(), student.avgGrade() + adjustment);  
            if (newStudent.avgGrade() >= minGrade) {  
                result.add(newStudent);  
            }  
        }  
    }  
}  
  
result.sort((student1, student2) -> Double.compare(student2.avgGrade(), student1.avgGrade()));  
return result;  
}
```

Optionale Zusatzaufgabe:

Die Durchschnittsnote darf nicht unter 1.0 oder über 6.0 fallen. Sie wird stattdessen auf die Grenzwerte gekappt.

Notenanpassung: Lösung

```
public List<Student> adjustedGrades(Collection<Student> students, double minGrade, int minAge, double adjustment) {  
    return students.stream()  
        .filter(student -> student.age() >= minAge)  
        .map(student -> new Student(student.name(), student.age(), student.avgGrade() + adjustment))  
        .filter(student -> student.avgGrade() >= minGrade)  
        .sorted((student1, student2) -> Double.compare(student2.avgGrade(), student1.avgGrade()))  
        .toList();  
}
```

Notenanpassung: Lösung (Bonus)

```
public static List<Student> adjustedGrades(Collection<Student> students, double minGrade, int minAge, double adjustment) {  
    return students.stream()  
        .filter(student -> student.age() >= minAge)  
        .map(student -> new Student(student.name(), student.age(), clamp(student.avgGrade() + adjustment, 1.0, 6.0)))  
        .filter(student -> student.avgGrade() >= minGrade)  
        .sorted((student1, student2) -> Double.compare(student2.avgGrade(), student1.avgGrade()))  
        .toList();  
}  
  
private static double clamp(double value, double min, double max) {  
    return Math.min(Math.max(value, min), max);  
}
```